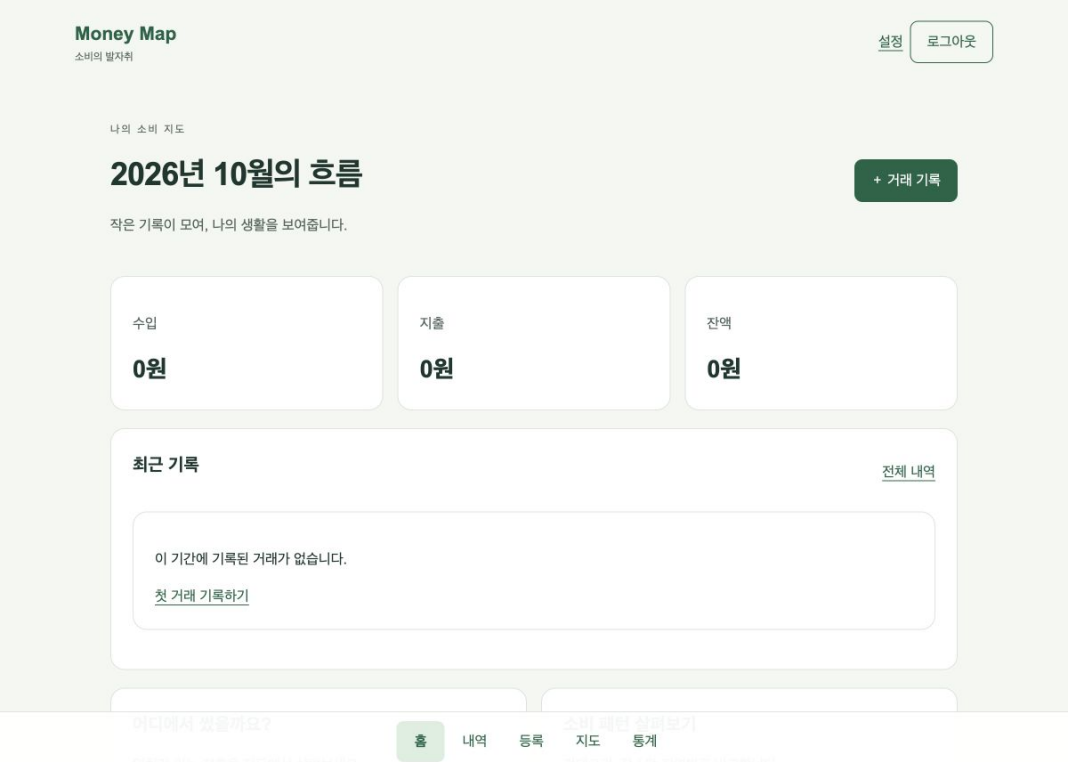
PERSONAL PROJECT / FULL-STACK / AI-ASSISTED

Money Map

AI와 함께, 요구사항에서 실제 배포까지

4년차 풀스택 개발자의 개인 프로젝트 · 2026.10.19 기준

지출에 선택적으로 위치를 연결하는 가계부를 만들고, 실제 HTTPS 환경에 배포해 본인 계정 로그인까지 확인했습니다. 자연어로 기능을 요청하는 데서 멈추지 않고, 계약·코드·실행 근거를 연결하며 AI의 결과를 검증했습니다.



실제 배포 서비스의 본인 로그인 직후 화면. 개인 거래가 없는 초기 상태이며, 실사용 규모를 보여주는 자료는 아닙니다.

범위	현재 결과
서비스	선택적 위치 기록 가계부 / 거래 · 지도 · 통계
완료 근거	코드·자동 검사·외부 서비스 연결·HTTPS 본인 로그인
운영 형태	개인 계정 사용 / 공개 회원가입 닫힘

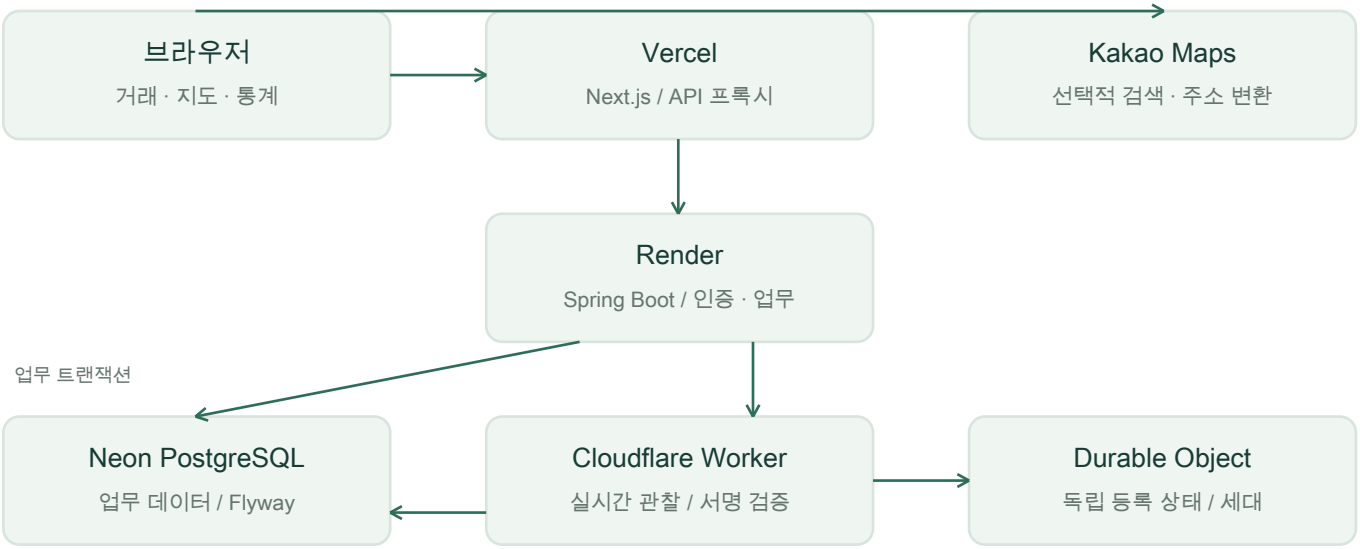
서비스 https://money-map-frontend-murex.vercel.app/
코드 https://github.com/Jimmy0613/money-map

02 / MONEY MAP

서비스와 기술의 연결

프런트·백엔드·데이터·외부 SDK를 하나의 사용자 흐름으로 구성했습니다.

로그인 → 금액·카테고리·날짜 입력 → 선택적 위치 확정 → 내역·지도·통계 확인. 위치 권한이나 지도 SDK가 실패하더라도 위치 없이 거래를 기록할 수 있는 경로를 유지했습니다.



구간	기술	적용 의도
프런트	Next.js 16 / React 19 / TypeScript	라우팅·모바일 화면·동일 Origin API 프록시
상태·폼	TanStack Query / React Hook Form / Zod	서버 상태·입력·검증 책임 분리
백엔드	Java 21 / Spring Boot 3.5 / JDBC	인증·업무 규칙·트랜잭션과 집계 SQL
DB·지도	PostgreSQL / Flyway / Kakao SDK	스키마 이력·데이터 제약·위치 스냅샷
배포	Vercel / Render Docker / Neon / Cloudflare	실행 특성에 맞는 호스트와 독립 개방 제어

보안·데이터 경계

HttpOnly 인증 쿠키, CSRF와 정확한 Origin 검사, Refresh Token 회전, 사용자별 소유권 검사를 적용했습니다. 앱에는 pooled DB 연결을, Flyway에는 direct 연결을 사용했습니다.

로컬 DB 통합 검사는 PostgreSQL 17.6, 실제 Neon은 18.6입니다. Flyway 지원 버전 경고를 기록했으며 완전한 호환 인증으로 표현하지 않습니다.

03 / MONEY MAP

바이브 코딩을 실행 가능한 과정으로

AI가 작성한 결과를 어떤 문서와 실행 근거로 확인했는지가 핵심입니다.

단계별 프롬프트와 승인 문서에서 출발해 요구사항·ERD·API 계약·체크리스트를 연결했습니다. 구현 후에도 문서를 다시 읽고 코드와 대조하는 점검을 요청하며 누락 항목을 보완했습니다.

단계	AI와 진행한 작업	남긴 결과
정의·설계	필수/선택 범위, 오류 계약, 데이터 제약 구체화	requirements / architecture / ERD / API spec
환경·구현	macOS·Docker 확인, 코드·마이그레이션 작성	environment-check / 소스 / Flyway
검토·보완	요구사항과 구현을 다시 대조하고 누락 보완	requirements-review / acceptance-criteria
실행 검증	Mock·실제 DB·브라우저·실기기 결과 구분	test-report / evidence
배포 연결	DB·서버·프런트·지도 도메인 설정과 실응답 확인	deployment-plan / 실제 로그인 화면

사람과 AI의 역할

사용자는 서비스 방향·사용 범위를 결정하고 승인 사항을 전달했습니다. 외부 계정 로그인, 권한 승인, 실기기 확인과 본인 계정 등록도 직접 수행했습니다. AI 코딩 에이전트는 문서 초안·설계 보완·코드 작성·테스트 실행·결함 탐색·배포 도구 구성을 수행했습니다.

AI와의 협업 범위를 공개하고, 작업을 실행 가능한 단위로 나눠 결과를 계약·테스트·서비스 상태와 연결한 경험을 보여주는 프로젝트입니다.

완료 판단의 기준

화면이 열리는 것, API가 응답하는 것, DB에 저장되는 것, 운영에 배포되는 것을 각각 확인했습니다. 전체 요구사항 충족과 개별 자동 테스트 통과도 같은 의미로 다루지 않았습니다.

04 / MONEY MAP

일관성을 지키는 프런트와 백엔드

정상 클릭 한 번보다 응답 유실·경합·지연 상태에 주목했습니다.

CASE A / 응답이 유실돼도 거래가 중복되지 않게

문제: 서버 저장은 성공했지만 응답만 유실되면 사용자의 재시도가 중복 거래를 만들 수 있습니다. 버튼 잠금만으로 서버의 처리 결과를 보장할 수는 없었습니다.

구현: Idempotency-Key와 정규화 본문 해시를 사용자별로 보관합니다. 사용자 행 잠금 후 거래·위치·작업 키를 같은 트랜잭션으로 저장하고 재요청을 판정합니다.

재요청	서버 처리
같은 키 + 같은 내용	기존 거래 반환
같은 키 + 다른 내용	409 충돌
삭제된 거래의 기존 작업 키	중복 재생성 거부

확인: 실제 DB 통합 검사와 이전 실제 브라우저 응답 유실 검사 기록이 있습니다. 대가: 사용자 행 잠금으로 같은 사용자의 동시 쓰기가 직렬화됩니다. 대규모 쓰기에 최적화했다고 주장하지 않습니다.

CASE B / 오래된 위치 응답이 최신 선택을 덮어쓰지 않게

문제: 장소 검색·현재 위치·주소 변환은 완료 순서가 다릅니다. 사용자가 다른 장소를 선택하거나 위치를 해제한 뒤 예전 응답이 돌아오면 선택이 뒤집힐 수 있습니다.

구현: 검색/선택 요청 순번과 현재 순번을 대조하고, 위치 변경·해제 시 과거 요청을 무효화합니다. 검색은 300ms debounce, 위치는 미리보기 후 확정하며 위치 없는 저장을 유지했습니다.

확인: 역순 응답·주소 변환 지연·위치 해제를 Frontend 테스트로 다룹니다. 외부 SDK 장애나 위치 오차 자체는 해결하지 못하므로 실패 안내와 대체 경로를 함께 구성했습니다.

코드 근거 LedgerService / LedgerRepository / shared/api.ts / location-picker.tsx
테스트 근거 PostgresIntegrationTest / location-races.test.tsx / test-report.md

05 / MONEY MAP

서버 기동과 운영 준비를 구분한 배포

보호 요구사항을 유지하며 정상 신규 DB를 정당하게 등록하는 경로를 만들었습니다.

문제: 무료 호스팅을 연결한 뒤 화면은 열리지만 API가 차단됐습니다. 기존 복원 보호 설계가 외부 검증 증거를 요구했고, 신규 빈 DB의 최초 등록 경로가 별도로 필요했습니다.

해결: 사용자 DB와 분리된 Cloudflare Durable Object에 등록 상태·세대를 저장했습니다. 최초 등록은 빈 DB·provider 결합·복원/reset 이력 부재를 확인한 뒤 한 번만 허용합니다. 일반 요청도 Neon 메타데이터와 SQL 관찰을 다시 수행합니다.

통제 지점	구현한 처리
최소 권한	개인정보 테이블을 읽지 못하는 메타데이터 함수 전용 DB 역할
요청 결합	대상 지문·환경·배포·nonce와 최대 5초 유효기간을 Ed25519로 서명
실패 대응	외부 검증 실패 시 차단. 복원·계보·스키마 변경은 BLOCKED 상태로 보존
배포 일치	Render 실제 commit과 등록된 배포를 대조

실제 확인한 결과

Render 상태 점검과 Vercel API 연결은 HTTPS 200으로 확인했습니다. 공개 회원가입은 허용 이메일도 403으로 거부했습니다. 본인은 loopback 전용 임시 화면에서 직접 계정을 만들었고, 임시 서버 종료 후 실제 HTTPS 서비스에서 로그인했습니다.

이 선택의 비용과 남은 경계

개인 MVP에 외부 검증 호출과 운영 복잡도를 추가한 선택입니다. 승인된 복원 차단 요구를 유지하기 위해 적용했으며, 모든 가계부 서비스에 필요한 일반 해법으로 보지는 않습니다. 최초 등록은 실제 WAL 복원·전체 재해 복구 완료를 뜻하지 않습니다. 이후 배포·스키마 변경의 등록 갱신 절차도 보완 대상입니다.

근거 bootstrap-deployment.md / RecoveryGate.java / infra/recovery-authority/src/

06 / MONEY MAP

결과는 확인한 범위만 말합니다

자동 테스트 사례 수를 기능 수나 전체 요구사항 수락률로 합산하지 않았습니다.

검증 층위	확인 결과	범위
Backend 단위·HTTP	53 PASS	최종 서버 빌드의 자동 검사
PostgreSQL 통합	25 PASS	Testcontainers 실제 DB
Worker 정책 / 런타임	5 + 4 PASS	서명·결합·SQLite·eviction·장애
Neon / Cloudflare	실제 등록·서명 확인	격리 환경 검사 및 별도 운영 최초 등록
운영 프로필 E2E	합성 가입·로그인·갱신·탈퇴	local 우회 없는 격리 DB + 실제 외부 검증
프런트 빌드	npm run build 성공	별도 복사본의 생산 빌드
공개 운영 서비스	API 연결·본인 로그인 확인	실제 HTTPS 응답과 대시보드

포함하지 않은 주장

본인이 입력한 첫 운영 거래의 저장·새로고침 검증은 진행하지 않았습니다. 과거 Frontend 55 PASS와 로컬 부하 기록은 현재 배포에서 모두 다시 실행한 수치로 더하지 않았습니다. 외부 검증 왕복이 추가된 현재 운영 성능으로 재사용하지도 않았습니다.

현재 운영 범위

본인용 계정으로 사용하는 개인 배포이며 공개 회원가입은 닫혀 있습니다. 별도 포트폴리오 시연 계정은 아직 없습니다. 서비스 링크는 로그인 화면 확인용으로 제공합니다.

다음 개선 우선순위

1. 운영 거래 저장·재조회와 현재 모바일 위치 흐름 확인
2. 개인 데이터와 분리된 시연 계정·샘플 데이터
3. 배포 등록 갱신과 콜드스타트·외부 검증 지연 관찰
4. 실제 백업·복원 훈련과 브라우저·접근성 검증 확장

07 / MONEY MAP

이 경험을 어떻게 설명할 것인가

AI 활용 자체보다, 연결과 검증의 과정을 보여주는 포트폴리오입니다.

4년차 풀스택 관점의 회고

AI로 화면과 API의 첫 구현을 확보한 뒤에도 문서 간 불일치, 미구현 항목, 운영 환경의 차이를 다시 확인해야 했습니다. 코드 생성 속도와 서비스 완성도를 구분하고, 요구사항 재대조·실패 시나리오·실제 배포 확인을 별도 단계로 유지했습니다.

프런트·백엔드·데이터·운영 사이의 경계를 연결하는 과정이 이 프로젝트의 핵심입니다. 동시에 개인 MVP에 추가한 제어 계층의 복잡도와 미완료 과제를 설명할 수 있도록, 적용 이유와 한계를 함께 남겼습니다.

60초 프로젝트 소개

“Money Map은 소비에 위치를 선택적으로 연결하는 가계부입니다. AI 코딩 에이전트와 함께 요구사항·설계·구현·테스트를 진행하고 Next.js, Spring Boot, PostgreSQL을 실제 HTTPS 환경에 배포했습니다. 핵심 사례는 응답 유실 시 중복 거래를 막는 멱등성 처리, 오래된 위치 응답을 버리는 프런트 상태 관리, 신규 운영 DB를 별도로 검증해 개방한 배포 과정입니다. AI의 코드 작성을 활용하면서 문서와 테스트·실행 증거로 결과를 확인한 과정을 보여주고 싶습니다.”

검토할 수 있는 자료

목적	저장소 근거
계약과 설계	requirements.md / architecture.md / erd.md / api-spec.md
재검토와 결과	requirements-review.md / acceptance-criteria.md / test-report.md
배포와 운영 경계	deployment-plan.md / bootstrap-deployment.md
실행 근거	evidence/bootstrap-*.log / 실제 배포·로그인 확인 기록

서비스 https://money-map-frontend-murex.vercel.app/
저장소 https://github.com/Jimmy0613/money-map
배포 소스 9f7cb26 · 작성 기준 2026.10.19

Secret·개인 이메일·비밀번호·정밀 위치·개인 거래 내역을 문서에 넣지 않았습니다. 개발 기간·절감 시간·사용자 수·팀 성과는 근거 없는 수치로 보완하지 않았습니다. 최신 로그인 증거 일부는 로컬에 보관 중이며 문서 갱신을 위한 임의 재배포는 하지 않았습니다.